



Amlogic

ADLA NN SDK API 描述

Revision: 1.3.0

Release Date: 2024-07-05

Confidential!
nick@khaos.com
2024-11-20 08:49:12

Copyright

© 2024 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

 , and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

更改记录

版本 1.3 (2024-07-05)

Section	Change Description
3.2.2	新增对 TENSOR_RAW_DATA 的支持，去掉 QTENSOR_RAW_DATA，更新图表
3.5	指明 profile 数据中的关键信息，描述如何通过设置环境变量来 dump op 文件
3.6	新增 online 流程的介绍及 demo

版本 1.2 (2023-10-20)

Section	Change Description
2	删除了 ADLA 平台不支持的接口：aml_util_setProfile、aml_util_setPowerPolicy、aml_util_getHardwareStatus
2.1.7	完善了 aml_util_getProfileInfo 接口的使用方式
3.2	注明了 ADLA 平台目前只支持做好前处理之后的 BINARY_RAW_DATA 输入模式
3.5	完善 aml_util_enableProfile 相关接口

版本 1.1 (2023-06-02)

Section	Change Description
1	更新简介描述

版本 1.0 (2022-10-20)

第一版。

目录

更改记录.....	ii
目录.....	iii
1. 简介	1
2. SDK API 及数据结构介绍	4
2.1 Amlogic SDK API 详细说明	4
2.1.1 aml_module_create.....	4
2.1.2 aml_module_input_set.....	5
2.1.3 aml_module_output_get	5
2.1.4 aml_module_output_get_simple.....	6
2.1.5 aml_module_destroy.....	6
2.1.6 aml_util_enableProfile.....	6
2.1.7 aml_util_getProfileInfo	7
2.1.8 aml_util_disableProfile	8
2.1.9 aml_read_chip_info.....	9
2.1.10 aml_util_mallocBuffer.....	9
2.1.11 aml_util_freeBuffer.....	10
2.1.12 aml_util_swapExternalInputBuffer	10
2.1.13 aml_util_swapExternalOutputBuffer	11
2.1.14 aml_util_getTensorInfo	11
2.1.15 aml_util_freeTensorInfo	12
2.2 Amlogic SDK 数据结构详细说明	12
2.2.1 aml_config.....	12
2.2.2 assign_user_address_t.....	14
2.2.3 aml_output_config_t.....	15
2.2.4 aml_perf_mode_t.....	15
2.2.5 aml_output_order_t.....	16
2.2.6 aml_output_format_t.....	16
2.2.7 image_out_t	17
2.2.8 tensor_info.....	17
2.2.9 info.....	18
2.2.10 detBox	18
2.2.11 point_t.....	19

2.2.12 nn_output	19
2.2.13 outBuf_t.....	19
2.2.14 nn_buffer_params_t.....	20
2.2.15 nn_input.....	21
2.2.16 aml_profiling_data_t.....	22
2.2.17 aml_profile_config_t.....	23
2.2.18 aml_memory_config_t.....	23
2.2.19 aml_memory_data_t	24
3. SDK API 使用方法介绍	25
3.1 SDK 自定义模型调用流程.....	25
3.2 SDK 基本调用流程说明	25
3.2.1 模型初始化.....	25
3.2.2 设置输入	26
3.2.3 获取模型结果	27
3.3 SDK 拓展功能介绍.....	28
3.3.1 SDK - DMA 功能介绍	28
3.3.2 SDK - usb camera 功能介绍	30
3.3.3 ffmpeg 解析视频接口介绍	31
3.4 参考代码.....	32
3.5 模型性能 debug 接口	34
3.6 online 流程介绍.....	35

1. 简介

Amlogic SDK 是一套已经集成多个常用模型，可以提供接口给客户直接调用并获取对应结果，且支持客户添加自定义模型的开发包。

本文介绍 Amlogic SDK API 接口以及相关的数据结构，用以指导 amlogic 相关开发人员以及 amlogic 客户。

```
Amlogic SDK 路径:          adla-toolkit-binary-
1. x. x. x/bin/adla_plug_in/template/nnsdk
```

```
Amlogic SDK Demo 路径:
adla-toolkit-binary-1. x. x. x/bin/adla_plug_in/template/example
```

SDK API 接口信息如表 1.1 所示，SDK API 数据结构信息如表 1.2 所示。

第一章：简介，介绍本文档的结构信息和主要内容。

第二章：SDK API 及数据结构介绍，详细介绍 SDK API 的功能、参数及返回值；详细介绍 SDK 定义的数据结构。

第三章：SDK API 使用方法介绍，介绍 SDK API 的使用方式并附上一段示例代码以供参考。

表 1.1 SDK API 接口表

SDK API	void* aml_module_create(aml_config* config);
	int aml_module_input_set(void* context,nn_input *pInput);
	void* aml_module_output_get(void* context,aml_output_config_t outconfig)
	void* aml_module_output_get_simple(void* context);
	int aml_module_destroy(void* context);
	int aml_util_getTensorInfo(void *context, const char* model_data, tensor_info** in_tInfo, tensor_info** out_tInfo);
	int aml_util_freeTensorInfo(tensor_info* tinfo);
	int aml_util_swapExternalInputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
	int aml_util_swapExternalOutputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
	int aml_util_mallocBuffer(void* context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);

	int aml_util_freeBuffer(void* context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
	int aml_util_enableProfile(void *context, aml_profile_config_t* profile_data);
	int aml_util_getProfileInfo(void *context, aml_profile_config_t* profile_data);
	int aml_util_disableProfile(void *context, aml_profile_config_t* profile_data);
	int aml_read_chip_info(void);

表 1.2 SDK API 数据结构表

结构体	功能
aml_config	输出 config 信息
assign_user_address_t	设置用户定义内存地址
aml_output_config_t	设置输出模型及数据类型信息
aml_output_format_t	设置输出数据类型
amlInn_input_mode_t	设置输入模式
amlInn_model_type	设置框架类型
amlInn_nbg_type	设置 nbg 文件读取方式
amlInn_input_type	设置输入数据类型
amlInn_query_cmd	设置 query 类型
nn_buffer_format_e	设置输出 buffer 类型
nn_buffer_quantize_format_e	设置量化类型
aml_module_t	设置 demo 类型
aml_profile_type_t	设置 profile 类型
aml_policy_type_t	设置 policy 类型

aml_flush_type_t	设置 flush tensor 类型
assign_user_address_t	设置分配地址类型
image_out_t	输出图像结果
tensor_info	输出 tensor 信息
nn_input	设置模型输入参数
aml_output_config_t	设置模型输出 config
aml_memory_config_t	设置内存类型参数
aml_profile_config_t	设置 profile 类型参数

2. SDK API 及数据结构介绍

2.1 Amlogic SDK API 详细说明

```
void* aml_module_create(aml_config* config);

int aml_module_input_set(void* context, nn_input *pInput);

void* aml_module_output_get(void* context, aml_output_config_t outconfig);

void* aml_module_output_get_simple(void* context);

int aml_module_destroy(void* context);

int aml_util_mallocBuffer(void* context, aml_memory_config_t* mem_config,
aml_memory_data_t* mem_data);

int aml_util_freeBuffer(void* context, aml_memory_config_t* mem_config,
aml_memory_data_t* mem_data);

int aml_util_swapExternalInputBuffer(void *context, aml_memory_config_t*
mem_config, aml_memory_data_t* mem_data);

int aml_util_swapExternalOutputBuffer(void *context, aml_memory_config_t*
mem_config, aml_memory_data_t* mem_data);

int aml_util_getTensorInfo(void *context, const char* model_data, tensor_info**
in_tInfo, tensor_info** out_tInfo);

int aml_util_freeTensorInfo(tensor_info* tinfo);

int aml_util_setProfile(aml_profile_type_t type, const char *savepath);

int aml_util_setPowerPolicy(aml_policy_type_t type);

int aml_util_getHardwareStatus(int* customID, int *powerStatus, int* version);

int aml_util_setAutoSuspend(int timeout);

int aml_util_enableProfile(void *context, aml_profile_config_t* profile_data);

int aml_util_getProfileInfo(void *context, aml_profile_config_t* profile_data);

int aml_util_disableProfile(void *context, aml_profile_config_t* profile_data);

int aml_read_chip_info(aml_platform_info_t* platform_info);
```

2.1.1 aml_module_create

API	void* aml_module_create(aml_config* config);
功能	初始化句柄，加载生成的 adla 模型
参数	aml_config* config: adla 模型的路径及类型信息
返回值	void*: 句柄 context 对象指针

示例代码：

```
void *context = NULL;
aml_config config;
memset(&config,0,sizeof(aml_config));

config.path = nb_name;
config.nbgType = NN_ADLA_FILE;
config.typeSize = sizeof(aml_config);
config.modelType =ADLA_LOADABLE;

context = aml_module_create(&config);
```

2.1.2 aml_module_input_set

API	int aml_module_input_set(void* context,nn_input *pInput);
功能	根据 adla 模型设置输入数据
参数	void* context: 句柄 context 指针
	nn_input *pInput: input 数据，当前单次只能设置一个 input，如果是多输入模型，暂时需要调用多次 amlnn_inputs_set 接口，后面会修正。
返回值	int: 成功返回零，失败返回非零。

示例代码：

```
nn_input inData;
unsigned char *rawdata;

rawdata = get_jpeg_rawData(jpeg_path,224,224);
inData.input_index = 0; //this value is index of input,begin from 0
inData.size = 224*224*3;
inData.input = rawdata;
inData.input_type = RGB24_RAW_DATA;
ret = aml_module_input_set(context,&inData);
```

2.1.3 aml_module_output_get

API	void* aml_module_output_get(void* context,aml_output_config_t outconfig)
功能	获取一次推理的结果
参数	void* context: 句柄 context 指针
	aml_output_config_t outconfig:模型类型及输出类型

返回值	void*: 推理结果结构体指针
-----	------------------

示例代码:

```
nn_output *outdata = NULL;
modelType = CUSTOM_NETWORK;
outconfig.format = AML_OUTDATA_FLOAT32;
outdata = (nn_output*)aml_module_output_get(context,outconfig);
process_top5((float*)outdata->out[0].buf,outdata->out[0].size/sizeof(float),NULL);
```

2.1.4 aml_module_output_get_simple

API	void* aml_module_output_get_simple(void* context);
功能	获取一次推理的结果，输出类型为 float32
参数	void* context: 句柄 context 指针
返回值	void*: 推理结果结构体指针

示例代码:

```
nn_output *outdata = NULL;
outdata = (nn_output *)aml_module_output_get_simple(qcontext);
process_top5((float*)outdata->out[0].buf,outdata->out[0].size/sizeof(float),NULL);
```

2.1.5 aml_module_destroy

API	int aml_module_destroy(void* context);
功能	卸载 nbg 模型并销毁 context 以及相关资源
参数	void* context: 句柄 context 指针
返回值	int: 成功返回零，失败返回非零。

示例代码:

```
int ret = aml_module_destroy(context);
```

2.1.6 aml_util_enableProfile

API	int aml_util_enableProfile(void *context, aml_profile_config_t* profile_data);
-----	--

功能	profile 性能调试功能使能
参数	void *context:句柄 context 指针。
	aml_profile_config_t* profile_data: 性能参数信息结构体
返回值	int: 成功返回零，失败返回非零。

示例代码：

```
context = aml_module_create(&config);
aml_profile_config_t profile_data;
aml_util_enableProfile(context, &profile_data);
```

2.1.7 aml_util_getProfileInfo

API	int aml_util_getProfileInfo(void *context, aml_profile_config_t* profile_data);
功能	获取 profile 性能参数信息
参数	void *context:句柄 context 指针。
	aml_profile_config_t* profile_data:性能参数信息结构体
返回值	int: 成功返回零，失败返回非零。

示例代码：

```
context = aml_module_create(&config);
aml_profile_config_t profile_data;

aml_util_enableProfile(context, &profile_data);

aml_util_getProfileInfo(context, &profile_data);
fprintf(stdout, "\nProfiling data:\n");
fprintf(stdout, "model_name: %s\n", config.path);

fprintf(stdout, "-----From system-----\n");
fprintf(stdout, "  core_freq_cur      : %llu Hz\n",
profile_data.profiling_data.ext.core_freq_cur);
fprintf(stdout, "  axi_freq_cur       : %llu Hz\n",
profile_data.profiling_data.ext.axi_freq_cur);
fprintf(stdout, "  mem_pool_size      : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_pool_size);
fprintf(stdout, "  mem_pool_used      : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_pool_used);
fprintf(stdout, "  mem_allocated_base : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_allocated_base);
fprintf(stdout, "  mem_allocated_umd  : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_allocated_umd);
int elapsed_hw_sw =
```

```

(int)profile_data.profiling_data.ext.us_elapsed_in_hw_op +
(int)profile_data.profiling_data.ext.us_elapsed_in_sw_op;

fprintf(stdout, "-----aml need-----\n");
fprintf(stdout, " : %7.2f ms, inference_time \n",
(float)profile_data.profiling_data.inference_time_us / 1000);
fprintf(stdout, " : %7.2f ms, elapsed_in_hw_op \n",
(float)profile_data.profiling_data.ext.us_elapsed_in_hw_op / 1000);
fprintf(stdout, " : %7.2f ms, elapsed_in_sw_op \n",
(float)profile_data.profiling_data.ext.us_elapsed_in_sw_op / 1000);
fprintf(stdout, " : %7.2f ms, elapsed (hw_op+sw_op) \n",
(float)elapsed_hw_sw/1000);
fprintf(stdout, " : %7.2f ms, invoke_time \n", (float)usVal_inference /
1000);
fprintf(stdout, " : %7.2f FPS(pm) \n", (double)1000000 /
(double)profile_data.profiling_data.inference_time_us);
fprintf(stdout, " : %7.2f FPS(hw+sw) \n", (double)1000000 /
(double)(elapsed_hw_sw));
fprintf(stdout, " : %7.2f FPS(invoke) \n", (double)1000000 /
(double)(usVal_inference));
fprintf(stdout, " : %7.2f Mbytes, dram_read_size
\n", (float)profile_data.profiling_data.dram_read_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, dram_write_size
\n", (float)profile_data.profiling_data.dram_write_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, BW(Ddr_data)
\n", (double)(profile_data.profiling_data.dram_read_bytes +
profile_data.profiling_data.dram_write_bytes) /
(1024 * 1024));

fprintf(stdout, " : %7.2f Mbytes, sram_read_bytes
\n", (float)profile_data.profiling_data.sram_read_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, sram_write_bytes
\n", (float)profile_data.profiling_data.sram_write_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes, BW(Sram_data)
\n", (double)(profile_data.profiling_data.sram_read_bytes +
profile_data.profiling_data.sram_write_bytes) /
(1024 * 1024));

fprintf(stdout, " : %7.2f Mbytes, memory_usage_bytes
\n", (float)profile_data.profiling_data.memory_usage_bytes / (1024 * 1024));

aml_util_disableProfile(context, &profile_data);

```

2.1.8 aml_util_disableProfile

API	int aml_util_disableProfile(void *context, aml_profile_config_t* profile_data);
功能	profile 性能调试功能关闭
参数	void *context:句柄 context 指针。
	aml_profile_config_t* profile_data:性能参数信息结构体

返回值	int: 成功返回零, 失败返回非零。
-----	---------------------

示例代码:

```
context = aml_module_create(&config);
aml_profile_config_t profile_data;
aml_util_disableProfile(context, &profile_data);
```

2.1.9 aml_read_chip_info

API	int aml_read_chip_info(aml_platform_info_t* platform_info);
功能	读取平台 NPU 硬件信息
返回值	int: 成功返回零, 失败返回非零。

示例代码:

```
aml_platform_info_t platform_info;
aml_read_chip_info(&platform_info);
Printf("sdk_version:%s\n",profile_data.sdk_version);
```

2.1.10 aml_util_mallocBuffer

API	int aml_util_mallocBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
功能	为 inbuf 或 outbuf 申请内存空间,以使用 input/output DMA 功能。
参数	void *context: 句柄 context 指针。
	aml_memory_config_t* mem_config,: 内存分配配置信息。
	aml_memory_data_t* mem_data: 驱动分配的内存信息。
返回值	int: 成功返回零, 失败返回非零。

示例代码:

```
aml_memory_config_t mem_config;
aml_memory_data_t mem_data;
mem_config.index = 0;
mem_config.mem_size = inData.size;
aml_util_mallocBuffer(context, &mem_config, &mem_data);

rawdata = get_jpeg_rawData(input_path, 224, 224);
memcpy(mem_data.viraddr, rawdata, inData.size);
```

```
ret = aml_module_input_set(qcontext, &inData);
```

2.1.11 aml_util_freeBuffer

API	int aml_util_freeBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
功能	释放使用 aml_util_mallocBuffer 分配的 inbuf dma 或 outbuf dma 内存空间。
参数	void *context: 句柄 context 指针。
	aml_memory_config_t* mem_config,: 内存释放配置信息。
	aml_memory_data_t* mem_data: 待释放内存信息。
返回值	int: 成功返回零, 失败返回非零。

示例代码:

```
aml_memory_config_t mem_config;
aml_memory_data_t mem_data;
aml_util_mallocBuffer(context, &mem_config, &mem_data);
```

2.1.12 aml_util_swapExternalInputBuffer

API	int aml_util_swapExternalInputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
功能	修改输入物理地址对应 DMA buffer 的起始指针, 实现 input 的快速切换
参数	void *context: 句柄 context 指针。
	aml_memory_config_t* mem_config,: 内存切换配置信息。。
	aml_memory_data_t* mem_data: 待切换内存信息。。
返回值	int: 成功返回零, 失败返回非零。

示例代码:

```
aml_memory_config_t mem_config;
aml_memory_data_t mem_data;

mem_config.index = 0;
mem_config.mem_size = inData.size;
```

```
rawdata = get_jpeg_rawData(input_path, 224, 224);
memcpy(mem_data.viraddr, rawdata, inData.size);

aml_util_swapExternalIoBuffer(context, mem_config, mem_data);
```

2.1.13 aml_util_swapExternalOutputBuffer

API	int aml_util_swapExternalOutputBuffer(void *context, aml_memory_config_t* mem_config, aml_memory_data_t* mem_data);
功能	修改输出物理地址对应 DMA buffer 的起始指针，实现 output 的快速切换
参数	void *context: 句柄 context 指针。
	aml_memory_config_t* mem_config,: 内存切换配置信息。
	aml_memory_data_t* mem_data: 待切换内存信息。
返回值	int: 成功返回零，失败返回非零。

2.1.14 aml_util_getTensorInfo

API	int aml_util_getTensorInfo(void *context, const char* model_data, tensor_info** in_tInfo, tensor_info** out_tInfo);
功能	获取 adla 文件的输入输出信息。
参数	void *context: 句柄 context 指针。
	const char* model_data,: adla 文件数据信息。
	tensor_info** in_tInfo: adla 中 input tensor 信息的指针。
	tensor_info** out_tInfo: adla 中 output tensor 信息的指针。
返回值	int: 成功返回零，失败返回非零。

示例代码：

```
tensor_info* in_tInfo = NULL;
tensor_info* out_tInfo = NULL;
aml_util_getTensorInfo(context, config.pdata,& in_tInfo, &out_tInfo);
{
    printf("the input buffer info is get success\n");
    printf("the input tensor number is %d\n",inptr->num);
}
```



```
printf("the input[0] data_format:%d\n",inptr->info[0].data_format);
printf("the input[0] quantization_format:%d\n",inptr->info[0].quantization_format);
printf("the input[0] sizes_of_dim:%d,%d,%d,%d\n",inptr->info[0].sizes_of_dim[0],
inptr->info[0].sizes_of_dim[1],inptr->info[0].sizes_of_dim[2],inptr->info[0].sizes_o
f_dim[3]);
}
aml_util_freeTensorInfo(in_tInfo);
aml_util_freeTensorInfo(out_tInfo);
```

2.1.15 aml_util_freeTensorInfo

API	int aml_util_freeTensorInfo(tensor_info* tinfo);
功能	释放分配的 inptr 或 outptr 的内存空间
参数	tensor_info*: getTensorInfo 获取的 inptr 或 outptr 数据
返回值	int: 成功返回零，失败返回非零。

示例代码：

```
tensor_info* in_tInfo = NULL;
tensor_info* out_tInfo = NULL;
aml_util_getTensorInfo(context, config.pdata,&in_tInfo, &out_tInfo);
{
    printf("the input buffer info is get success\n");
    printf("the input tensor number is %d\n",inptr->num);
    printf("the input[0] data_format:%d\n",inptr->info[0].data_format);
    printf("the input[0] quantization_format:%d\n",inptr->info[0].quantization_format);
    printf("the input[0] sizes_of_dim:%d,%d,%d,%d\n",inptr->info[0].sizes_of_dim[0],
inptr->info[0].sizes_of_dim[1],inptr->info[0].sizes_of_dim[2],inptr->info[0].sizes_o
f_dim[3]);
}
aml_util_freeTensorInfo(in_tInfo);
aml_util_freeTensorInfo(out_tInfo);
```

2.2 Amlogic SDK 数据结构详细说明

2.2.1 aml_config

成员变量	数据类型	含义
typeSize	int	结构体成员变量数
path	const char *	load nbg from file 指针
pdata	const char *	load nbg from memory 指针

length	int	nbg size
modelType	amlInn_model_type	nbg 模型类型
nbgType	amlInn_nbg_type	load nbg 方式
inOut	assign_user_address_t	分配用户地址
forward_ctrl	aml_forward_ctrl_t	模型推理控制选项
compiler_args	aml_compiler_args_t	模型编译控制选项

```
typedef struct __aml_nn_config
```

```
{
```

```
    int typeSize;
```

```
    int length;
```

```
    const char *path;
```

```
    const char *pdata;
```

```
    amlInn_model_type modelType;
```

```
    amlInn_nbg_type nbgType;
```

```
    assign_user_address_t inOut;
```

```
    aml_forward_ctrl_t forward_ctrl;
```

```
    aml_compiler_args_t compiler_args;
```

```
} aml_config;
```

```
typedef enum _amlInn_nbg_type_ {
```

```
    NN_NBG_FILE = 0,
```

```
    NN_NBG_MEMORY = 1,
```

```
    NN_RUNTIME_FILE = 2,
```

```
    NN_RUNTIME_MEMORY = 3,
```

```
    NN_ADLA_FILE = 4,
```

```
    NN_ADLA_MEMORY = 5
```

```
} amlInn_nbg_type;
```

```
typedef struct __aml_forward_ctrl_t
```

```
{
```

```
    aml_encore_id          enCoreId;      /* device target which running the  
seg*/
```

```
    int64_t                invoke_id;
```

```
        int32_t          timeout_ms;
    } aml_forward_ctrl_t;
```

```
typedef struct __aml_compiler_args_t
{
    int32_t batch_multiplier;
    int32_t compiler_only;
} aml_compiler_args_t;
```

amlInn_model_type

成员变量	模型类型
CAFFE	Caffe 模型
TENSORFLOW	Tensorflow 模型
TENSORFLOWLITE	TensorflowLite 模型
DARKNET	Darknet 模型
ONNX	Onnx 模型
KERAS	Keras 模型
PYTORCH	Pytorch 模型
ADLA_LOADABLE	Adla 模型

2.2.2 assign_user_address_t

成员变量	数据类型	含义
io_type	aml_io_format_t	设置 dma 类型
inAddr	unsigned char*	输入地址
outAddr	unsigned char*	输出地址

```
typedef struct __assign_address
{
```

```
    unsigned char* inAddr[ADDRESS_MAX_NUM];
    unsigned char* outAddr[ADDRESS_MAX_NUM];
}assign_user_address_t;
typedef enum {
    AML_IO_VIRTUAL      = 0,
    AML_IO_PHYS         = 1,
} aml_io_format_t;
```

2.2.3 aml_output_config_t

成员变量	数据类型	含义
typeSize	int	结构体成员变量数
mdType	aml_module_t	网络模型
perfMode	aml_perf_mode_t	前向推理测试模式
format	aml_output_format_t	输出数据类型
order	aml_output_order_t	输出数据排列方式，NCHW/NHWC

```
typedef struct __amlnn_module_out_data_t
{
    int typeSize;
    aml_module_t mdType;
    aml_perf_mode_t perfMode;
    aml_output_format_t format;
    aml_output_order_t order;
} aml_output_config_t;
```

2.2.4 aml_perf_mode_t

成员变量	含义
AML_NO_PERF	默认不开启测试模式

AML_PERF_INFERENCE	测试模式下只进行前向推理
AML_PERF_OUTPUT	测试模式下只进行反量化

```
typedef enum {  
    AML_NO_PERF            = 0,  
    AML_PERF_INFERENCE     = 1,  
    AML_PERF_OUTPUT        = 2  
} aml_perf_mode_t;
```

2.2.5 aml_output_order_t

成员变量	含义
AML_OUTPUT_ORDER_DEFAULT	adla 以 NHWC 的顺序输出,galcore 以 NCHW 的顺序输出
AML_OUTPUT_ORDER_NHWC	以 NHWC 的顺序输出
AML_OUTPUT_ORDER_NCHW	以 NCHW 的顺序输出

```
typedef enum {  
    AML_OUTPUT_ORDER_DEFAULT = 0,    //output format:  nhwc for adla,  
    nchw for galcore  
    AML_OUTPUT_ORDER_NHWC    = 1,    //output format: nhwc  
    AML_OUTPUT_ORDER_NCHW    = 2,    //output format: nchw  
} aml_output_order_t;
```

2.2.6 aml_output_format_t

成员变量	含义
AML_OUTDATA_FLOAT32	输出 float32 类型的数据
AML_OUTDATA_RAW	输出原始数据
AML_OUTDATA_DMA	输出 dma 数据

```
typedef enum {  
    AML_OUTDATA_FLOAT32 = 0,  
    AML_OUTDATA_RAW     = 1,
```

```
    AML_OUTDATA_DMA          = 2
} aml_output_format_t;
```

2.2.7 image_out_t

成员变量	数据类型	含义
height	int	输出图像高度值
width	int	输出图像宽度值
channel	int	输出图像通道数
data	unsigned char *	输出图像 buffer 信息

```
typedef struct __nn_image_out
{
    int height;
    int width;
    int channel;
    unsigned char *data; //this buffer is returned by aml_module_output_get
}image_out_t;
```

2.2.8 tensor_info

成员变量	数据类型	含义
valid	unsigned int	获取 tensor 信息 flag
num	unsigned int	tensor 数目
info	info *	tensor 信息指针

```
typedef struct {
    unsigned int valid;
    unsigned int num;
    info *info;
} tensor_info;
```

2.2.9 info

成员变量	数据类型	含义
dim_count	unsigned int	维度数目
sizes_of_dim	unsigned int[]	维度值
data_format	unsigned int	buffer format 类型
data_type	unsigned int	数据类型
quantization_format	unsigned int	量化类型
fixed_point_pos	int	int8/int16 量化的参数值
TF_scale	float	scale 值
TF_zerpoint	int	zerpoint 值
name	char[]	名称

```
typedef struct {  
    unsigned int dim_count;          /*dim count*/  
    unsigned int sizes_of_dim[4]; /*dim value,just support 4-d dim*/  
    unsigned int data_format;        /*see as nn_buffer_format_e*/  
    unsigned int data_type;          /*not use*/  
    unsigned int quantization_format; /*see as nn_buffer_quantize_format_e*/  
    int fixed_point_pos;              /*for int8/int16 UANTIZE_DYNAMIC_FIXED_POINT*/  
    float TF_scale;                   /*as tf define,scale*/  
    int TF_zerpoint;                  /*as tf define,zerpoint*/  
    char name[MAX_NAME_LENGTH];      /*not use,will used in future*/  
} info
```

2.2.10 detBox

成员变量	数据类型	含义
x	float	输出目标框的 X 轴坐标值
y	float	输出目标框的 Y 轴坐标值

w	float	输出目标框的宽度信息
h	float	输出目标框的高度信息
score	float	输出目标的得分信息
objectClass	float	输出目标的类别信息

2.2.11 point_t

成员变量	数据类型	含义
x	float	输出目标点的 X 轴坐标值
y	float	输出目标点的 Y 轴坐标值

2.2.12 nn_output

成员变量	数据类型	含义
typeSize	int	结构体成员变量数
num	unsigned int	输出 tensor 的个数
out	outBuf_t[]	输出 tensor 结构体数组

```
typedef struct __nnout
{
    int typeSize;
    unsigned int num; /*=====output tensor number=====*/
    outBuf_t out[OUTPUT_MAX_NUM];
}nn_output;
```

2.2.13 outBuf_t

成员变量	数据类型	含义
size	unsigned int	输出 tensor 的个数
buf	unsigned char *	输出 tensor 的数据地址

param	nn_buffer_params_t *	输出 tensor 的属性
name	char	输出 tensor name

```
typedef struct out_buf
{
    unsigned int size;
    unsigned char *buf;
    nn_buffer_params_t *param;
    char  name[MAX_NAME_LEGTH];    //output tensor name
    aml_output_format_t out_format;
}outBuf_t;
```

2.2.14 nn_buffer_params_t

成员变量	数据类型	含义
num_of_dims	unsigned int	Tensor 维度个数
sizes	unsigned int[]	Tensor 各维度值
data_format	nn_buffer_format_e	tensor 的数据类型，有以下几种： NN_BUFFER_FORMAT_FP32 = 0, NN_BUFFER_FORMAT_FP16 NN_BUFFER_FORMAT_UINT8 NN_BUFFER_FORMAT_INT8 NN_BUFFER_FORMAT_UINT16 NN_BUFFER_FORMAT_INT16
quant_format	nn_buffer_quantize_format_e	量化格式，有以下几种值： NN_BUFFER_QUANTIZE_NONE=0, NN_BUFFER_QUANTIZE_DYNAMIC_FIXED_POINT（动态定点量化） NN_BUFFER_QUANTIZE_TF_ASYMM（非对称量化）
dfp	struct	DYNAMIC_FIXED_POINT 量化类型参数： fl = dfp.fixed_point_pos

affine	struct	TF_ASYMM 量化类型参数: scale=affine.scale zeroPoint=affine.zeroPoint
--------	--------	--

```
typedef struct _nn_buffer_create_params_t
{
    unsigned int    num_of_dims;
    unsigned int    sizes[4];
    nn_buffer_format_e    data_format;
    nn_buffer_quantize_format_e quant_format;
    union {
        struct {
            unsigned char fixed_point_pos;
        } dfp;
        struct {
            float    scale;
            unsigned int zeroPoint;
        } affine;
    }
    quant_data;
} nn_buffer_params_t;
```

2.2.15 nn_input

成员变量	数据类型	含义
typeSize	int	结构体成员变量数
input_index	int	输入数据对应输入 tensor 的下标，单个输入为 0
size	int	输入数据大小
input	unsigned char*	输入数据地址
input_type	amlnn_input_type	输入数据类型，有以下几种类型： RGB24_RAW_DATA = 0, TENSOR_RAW_DATA,

		QTENSOR_RAW_DATA, BINARY_RAW_DATA, INPUT_DMA_DATA
info	input_info	输入 buffer 信息

```
typedef struct {
    int valid;
    float mean[INPUT_CNANNEL];
    float scale;
    aml_input_format_t input_format;
    int int16_type ; //when int16 quantize model mean value is not 128,128,128,set
1    int preprocess_debug; // save npu input data
}input_info;
typedef enum {
    AML_INPUT_DEFAULT      = 0,    //channle format: caffe 2 1 0 ,others 0 1
2    AML_INPUT_MODEL_NHWC   = 1,    //channle format: 0 1 2
    AML_INPUT_MODEL_NCHW   = 2,    //channle format: 2 1 0
} aml_input_format_t;
```

2.2.16 aml_profiling_data_t

成员变量	数据类型	含义
inference_time_us	int	推理耗时
memory_usage_bytes	int	内存占用总体情况
memory_read_bytes	int	读内存占用情况
memory_write_bytes	int	写内存占用情况
ext	aml_profiling_ext_data_t	其它性能信息

```
typedef struct aml_profiling_ext_data
{
    unsigned int axi_freq_cur;
    unsigned int core_freq_cur;
```

```
    unsigned int mem_allocated_base;
    unsigned int mem_allocated_umd;
    int mem_pool_size;
    unsigned int mem_pool_used;
    int us_elapsed_in_hw_op;
    int us_elapsed_in_sw_op;
} aml_profiling_ext_data_t;
```

2.2.17 aml_profile_config_t

成员变量	数据类型	含义
suspend_time	unsigned int	Context 内存占用信息
freq	float	Context 模型层数信息
profile_type	aml_profile_type_t	性能信息类型
policy_type	aml_policy_type_t	电源策略信息
profiling_data	aml_profiling_data_t	性能信息
frequency_data	aml_work_frequency_t	频率信息

```
typedef struct aml_work_frequency
{
    float *curr_freq;
    float *min_freq;
} aml_work_frequency_t;
```

2.2.18 aml_memory_config_t

成员变量	数据类型	含义
typeSize	int	结构体 size
mmu_type	float	MMU 状态
cache_type	aml_cache_type_t	Cache 模式
memory_type	aml_memory_type_t	内存状态

profiling_data	aml_profiling_data_t	性能信息
frequency_data	aml_work_frequency_t	频率信息

```
typedef struct __aml_memory_config_t
{
    uint32_t      typeSize;
    uint32_t      mem_size;
    uint32_t      index;
    aml_mmu_type_t mmu_type;
    aml_cache_type_t cache_type;
    aml_memory_type_t memory_type;
    aml_mem_direction_t direction;
} aml_memory_config_t;
```

2.2.19 aml_memory_data_t

成员变量	数据类型	含义
typeSize	int	结构体 size
memory	void*	Kernel 层虚拟地址
viraddr	void*	user 层虚拟地址
phyaddr	uint64_t	Kernel 层物理地址

```
typedef struct __aml_memory_data_t
{
    uint32_t      typeSize;
    void          *memory;
    void          *viraddr;
    uint64_t      phyaddr;
} aml_memory_data_t;
```

3. SDK API 使用方法介绍

目前 SDK 包使用特定网络 ID: CUSTOM_NETWORK 来代表新的网络, 这样输出的数据就会保留原始网络的输出, 不会针对该类型做后处理操作。在多网络应用时, 客户根据网络 aml_module_create 返回的 context 值来区分管理多个网络实例。

各功能使用方式示例 demo 请见 GitHub: https://github.com/Amlogic-NN/AML_NN_SDK/tree/master/Linux/Demo

3.1 SDK 自定义模型调用流程

1、初始化模型: void* context = aml_module_create(&config);

2、准备输入数据, 并设置输入数据:

```
inData.input_index = 0;
inData.size = 224*224*3;
inData.input = rawdata;
inData.input_type = INPUT_DMA_DATA;
int ret = aml_module_input_set(context,&inData);
```

3、进行前向推理及获取推理结果:

```
outdata = (nn_output *)aml_module_output_get_simple(context);
```

4、多次推理: 多次重复执行 6、7 两步。

5、卸载模型: int ret = aml_module_destroy(context);

3.2 SDK 基本调用流程说明

3.2.1 模型初始化

aml_config config;

1. 有两种模型 adla load 方式

方式一: load adla from memory

```
fp = fopen(argv[1],"rb");
fseek(fp,0,SEEK_END);
size = (int)ftell(fp);
rewind(fp);
config.pdata = (char *)calloc(1,size);
fread((void*)config.pdata,1,size,fp);
config.nbgType = NN_ADLA_MEMORY;
config.length = size;
fclose(fp);
```

方式二：load adla from file

```
config.path= (const char *)argv[1];  
config.nbgType = NN_ADLA_FILE;
```

2. 有两种模型 runtime load 方式

方式一：load runtime from memory

```
fp = fopen(argv[1],"rb");  
fseek(fp,0,SEEK_END);  
size = (int)ftell(fp);  
rewind(fp);  
config.pdata = (char *)calloc(1,size);  
fread((void*)config.pdata,1,size,fp);  
config.nbgType = NN_RUNTIME_MEMORY;  
config.length = size;  
fclose(fp);
```

方式二：load runtime from file

```
config.path= (const char *)argv[1];  
config.nbgType = NN_RUNTIME_FILE;
```

```
config.modelType = ADLA_LOADABLE;  
void *context = aml_module_create(&config);
```

注：

1、file 方式输入参数为 adla、runtime 文件的路径。其中，config.path 是指向 adla 具体信息的指针，当指定 config.nbgType 为 NN_RUNTIME_FILE、NN_ADLA_FILE 时，其赋值分别对应 runtime、adla 文件的路径。

2、当模型文件内容已保存在内存中时，可以选择从 memory 加载。其中，config.pdata 是指向 runtime、adla 具体信息的指针，当指定 config.nbgType 为 NN_***_MEMORY 时，其赋值代表模型文件的 memory 首地址。与 NN_ADLA_FILE 不同，该方式需额外设置 config.size 为模型文件的 memory 的实际长度。

3、使用 memory 加载模型文件的方式需要进行内存释放，free((void*)config.pdata)。

3.2.2 设置输入

```
nn_input inData;  
  
unsigned char *rawdata = read_file(input_path, &input_size); //读取输入数据  
  
inData.input_index = 0; //输入数据索引值，支持数据多输入  
  
inData.size = 416*416*3; //设置输入尺寸，W*H*C
```

```
inData.input = rawdata; //设置输入 buffer
inData.input_type = BINARY_RAW_DATA; //设置输入数据格式
```

注:

- 1、read_file: 读取做好前处理之后的输入文件。
- 2、inData.size: 根据实际模型设置输入尺寸, W*H*C 。
- 3、input_index: 输入数据索引值。单个输入设置 0 即可, 多个输入的情况, 需多次调用该接口, 索引值依次加 1。
- 4、inData.input_type: 输入数据类型:

输入数据类型	含义	处理操作
RGB24_RAW_DATA	输入RGB格式数据, (0, 255)	data/255/scale + zp
TENSOR_RAW_DATA	原始tensor数据float输入	data/scale + zp
BINARY_RAW_DATA	原始数据输入, 中间不做任何处理与转换	memcpy

此外支持 INPUT_DMA_DATA 类型, 如使用 input dma 功能, 则 inData.input_type 需要设置为该类型。

3.2.3 获取模型结果

使用 nn_output 的输出结构体来接收 aml_module_output_get 接口返回的结果。

```
nn_output *outdata = NULL;
outconfig.typeSize = sizeof(aml_output_config_t);
modelType = CUSTOM_NETWORK; //设置输出模型类型
outconfig.format = AML_OUTDATA_FLOAT32; //设置输出数据类型
outdata = (nn_output*)aml_module_output_get(context,outconfig);
if(outdata->out[0].param->data_format == NN_BUFFER_FORMAT_FP32)
{
    process_top5((float*)outdata->out[0].buf,outdata->out[0].size/sizeof(float),NULL);
}
```

outdata 中包含了完整的输出信息, 包括输出的 dim, 输出的格式, 输出的内容等等, 输出内容默认为 float32 格式, 后面可以进行相关网络的后处理实现, 如 mobilenet:

```
process_top5((float*)pout->out[0].buf,pout->out[0].size/sizeof(float));
```

注:

- 1、outconfig.format 可根据实际需要选择 AML_OUTDATA_FLOAT32、AML_OUTDATA_RAW、AML_OUTDATA_DMA。

2、其中 AML_OUTDATA_FLOAT32 表示输出的数据类型为 f32，类型转换已在 sdk 内部完成；AML_OUTDATA_RAW 表示输出原始类型的数据；AML_OUTDATA_DMA 表示输出 dma output buffer 数据类型，需要配合 DMA 使用。

3、为了便于用户使用自定义模型，提供一个参数简单的接口进行前向推理及获取推理结果。

```
void* aml_module_output_get_simple(void* context);
```

此接口只需传入 context，默认输出数据类型为 float32。其使用方式为：

```
nn_output * outdata = (nn_output *)aml_module_output_get_simple(context);
```

用户可根据需要灵活选择推理接口。

4、若用户需要获取模型前向推理时间，可通过 outconfig.perfMode 开启性能测试模式分别获取前向推理和反量化的耗时。outconfig.perfMode 默认不设置，测试模式默认关闭。

例如：

```
outconfig.typeSize = sizeof(aml_output_config_t);
modelType = CUSTOM_NETWORK;
outconfig.format = AML_OUTDATA_FLOAT32;
tmsStart = get_perf_count();
outconfig.perfMode = AML_PERF_INFERENCE;
for (i=0;i<100;i++)
    outdata = (nn_output*)aml_module_output_get(qcontext,outconfig);
tmsEnd = get_perf_count();
msVal = (tmsEnd - tmsStart)/1000000;
usVal = (tmsEnd - tmsStart)/1000;
printf("AML_PERF_INFERENCE: %ldms or %ldus\n", msVal, usVal);

tmsStart = get_perf_count();
outconfig.perfMode = AML_PERF_OUTPUT;
outdata = (nn_output*)aml_module_output_get(qcontext,outconfig);
tmsEnd = get_perf_count();
msVal = (tmsEnd - tmsStart)/1000000;
usVal = (tmsEnd - tmsStart)/1000;
printf("AML_PERF_OUTPUT: %ldms or %ldus\n", msVal, usVal);
```

3.3 SDK 拓展功能介绍

3.3.1 SDK - DMA 功能介绍

1、input dma 设置

1.1 Virtual Address

```
aml_memory_config_t mem_config;
```

```
aml_memory_data_t mem_data;

input_size = 224*224*3;

mem_config.mem_size = input_size;

mem_config.index = 0;

mem_config.memory_type = AML_VIRTUAL_ADDR;

aml_util_mallocBuffer(context, &mem_config, &mem_data);

rawdata = get_jpeg_rawData(input_path, 224, 224);

memcpy(mem_data.viraddr, rawdata, inData.size);

aml_util_swapExternalInputBuffer(context, mem_config, mem_data);
```

注：

- 1、aml_util_mallocBuffer 分配内存需对应输入图像的尺寸。

1.2 Physical Address

```
aml_memory_config_t mem_config;

aml_memory_data_t mem_data;

input_size = 224*224*3;

mem_config.mem_size = input_size;

mem_config.index = 0;

mem_config.memory_type = AML_PHYS_ADDR;;

aml_util_mallocBuffer(context, &mem_config, &mem_data);

rawdata = get_jpeg_rawData(input_path, 224, 224);

memcpy(mem_data.viraddr, rawdata, inData.size);

aml_util_swapExternalInputBuffer(context, mem_config, mem_data);
```

注：

- 1、ainput_index 为 inputId，从 0 开始。
- 2、该操作可以在第一帧执行过程中准备好，形成类似 ping-pang buffer 的效果，以省去 input 输入的时间。

2、output dma 设置

2.1 Virtual Address

```
aml_memory_config_t mem_config;

aml_memory_data_t mem_data;

mem_config.mem_size = output_size;

mem_config.index = 0;

mem_config.memory_type = AML_VIRTUAL_ADDR;;

aml_util_mallocBuffer(context, &mem_config, &mem_data);
```

```
aml_util_swapExternalOutputBuffer(context, mem_config, mem_data);
```

2.2 Physical Address

```
output_size = 2*1000;  
aml_memory_config_t mem_config;  
aml_memory_data_t mem_data;  
mem_config.mem_size = output_size;  
mem_config.index = 0;  
mem_config.memory_type = AML_PHYS_ADDR;;  
aml_util_mallocBuffer(context, &mem_config, &mem_data);  
aml_util_swapExternalOutputBuffer(context, mem_config, mem_data);
```

注

1、aml_util_mallocBuffer 需要设置输出 buffer size，如如分类模型的分类结果是 1000 个种类，每个种类是 float16 类型，故 outbuf 需要的长度为 1000*2。

3.3.2 SDK - usb camera 功能介绍

usb camera 通过调用 camera_thread_func 和 net_thread_func 两个线程实现实时显示在 framebuffer 上。

```
context = init_network(argc,argv);  
pthread_mutex_init(&mutex_data,NULL);  
init_fb();  
if (0 != pthread_create(&tid[0],NULL,camera_thread_func,NULL))  
{  
    fprintf(stderr, "Couldn't create thread func\n");  
    return -1;  
}  
thread_args = (void*)argv[3];  
if (0 != pthread_create(&tid[1],NULL,net_thread_func,thread_args))  
{  
    fprintf(stderr, "Couldn't create thread func\n");  
    return -1;  
}  
while(1)
```

```

{
    for (i=0;i<2;i++)
        pthread_join(tid[i], NULL);
}

```

其中，net_thread_func 中 camera 的调用方式可参考以下代码，详细代码请见 release 包中的 face_detect 模型实现。

```

dup_rgbbuf = (unsigned char*)malloc(display_width * display_high * 3);
input_data = (unsigned char*)malloc(input_width * input_high * 3);
while(1)
{
    if(rgbbuf != NULL)
    {
        pthread_mutex_lock(&mutex_data);
        memcpy(dup_rgbbuf,rgbbuf,display_width*display_high*3);
        pthread_mutex_unlock(&mutex_data);
    }
    resize_input_data(dup_rgbbuf,input_data);
    ret = run_network(netType,context,input_data,AML_IN_CAMERA,dup_rgbbuf);
}

```

注：

1、当前分辨率默认设置为 640*480，支持 1920*1080、1280*720、640*480 三种分辨率。

3.3.3 ffmpeg 解析视频接口介绍

当前 SDK 支持输入视频数据，利用 ffmpeg 解码视频用于网络的输入。将视频解析并经过 resize 得到的图像作为 rawdata 输入数据送入模型。

```

sprintf(cmd,"ffmpeg -i %s -qscale:v 2 -r 24 ",jpath);
ptr=strcat(cmd,"tmp/image%5d.bmp");
system(ptr);
sprintf(img_name,"tmp/image%05d.bmp",index);
while (1)
{
    sprintf(img_name,"tmp/image%05d.bmp",index);
    if((access(img_name,F_OK)) < 0)
    {

```

```

        break;
    }
    img=imread(img_name,199);
    resize(img,dst,dst.size());

    ret = run_network(netType,context,dst.data,AML_IN_VIDEO,(unsigned
char*)img_name);
    index++;
}

system("ffmpeg -f image2 -i tmp/image%5d.bmp -b:v 5626k videoout.mp4");

```

3.4 参考代码

```

int main(int argc,char **argv)
{
    void *context;
    const char *jpeg_path = NULL;
    unsigned char *rawdata;
    aml_config config;
    nn_input inData;
    nn_output *outdata = NULL;
    int ret;
    int i;
    nn_query *query;
    input_path = (const char *)argv[2];
    config.modelType = TENSORFLOW;
    /***** load adla from memory *****/
    fp = fopen(argv[1],"rb");
    fseek(fp,0,SEEK_END);
    size = (int)ftell(fp);
    rewind(fp);
    config.pdata = (char *)calloc(1,size);
    fread((void*)config.pdata,1,size,fp);
    config.nbgType = NN_ADLA_MEMORY;
    config.size = size;
    fclose(fp);
    /***** load adla from file *****/
    config.path= (const char *)argv[1];
    config.nbgType = NN_ADLA_FILE;

    context = aml_module_create(&config);
    rawdata = read_file(input_path, &input_size);
    inData.input_index = 0; //this value is index of input,begin from 0
    inData.size = 224*224*3;
    inData.input = rawdata;
    inData.input_type = BINARY_RAW_DATA;
    ret = aml_module_input_set(context,&inData);
    if(rawdata != NULL)
    {
        free(rawdata);
    }
}

```

```

        rawdata = NULL;
    }
    outdata = (nn_output *)aml_module_output_get_simple(context);
    if(outdata->out[0].param->data_format == NN_BUFFER_FORMAT_FP32)
    {
        process_top5((float*)outdata->out[0].buf,outdata->out[0].size/sizeof(float),NULL)
    }
    ret = aml_module_destroy(context);
    return ret;
}

```

outdata 中包含了完整的输出信息，包括输出的 dim，输出的格式，输出的内容等等，输出内容默认为 float32 格式，后面可以进行相关网络的后处理实现，如 mobilenet:

```

process_top5((float*)pout->out[0].buf,pout->out[0].size/sizeof(float));
void process_top5(float *buf,unsigned int num,img_classify_out_t* clsout)
{
    int j = 0;
    unsigned int MaxClass[5]={0};
    float fMaxProb[5]={0.0};

    float *pfMaxProb = fMaxProb;
    unsigned int *pMaxClass = MaxClass,i = 0;

    for (j = 0; j < 5; j++)
    {
        for (i=0; i<num; i++)
        {
            if ((i == *(pMaxClass+0)) || (i == *(pMaxClass+1)) || (i == *(pMaxClass+2)) ||
                (i == *(pMaxClass+3)) || (i == *(pMaxClass+4)))
            {
                continue;
            }
            if (buf[i] > *(pfMaxProb+j))
            {
                *(pfMaxProb+j) = buf[i];
                *(pMaxClass+j) = i;
            }
        }
    }
    for (i=0; i<5; i++)
    {
        if (clsout == NULL)
        {
            printf("%3d: %8.6f\n", MaxClass[i], fMaxProb[i]);
        }
        else
        {
            clsout->score[i] = fMaxProb[i];
            clsout->topClass[i] = MaxClass[i];
        }
    }
}

```

3.5 模型性能 debug 接口

```

aml_util_enableProfile(context, &profile_data);
ret = run_network(netType, context, jpath);
aml_util_getProfileInfo(context, &profile_data);

fprintf(stdout, "\nProfiling data:\n");
fprintf(stdout, "model_name: %s\n", config.path);

fprintf(stdout, " -----From system-----\n");
fprintf(stdout, "  core_freq_cur      : %llu Hz\n",
profile_data.profiling_data.ext.core_freq_cur);
fprintf(stdout, "  axi_freq_cur       : %llu Hz\n",
profile_data.profiling_data.ext.axi_freq_cur);
fprintf(stdout, "  mem_pool_size      : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_pool_size);
fprintf(stdout, "  mem_pool_used      : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_pool_used);
fprintf(stdout, "  mem_allocated_base : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_allocated_base);
fprintf(stdout, "  mem_allocated_umd  : 0x%08lX Bytes\n",
profile_data.profiling_data.ext.mem_allocated_umd);
int elapsed_hw_sw =
    (int)profile_data.profiling_data.ext.us_elapsed_in_hw_op +
    (int)profile_data.profiling_data.ext.us_elapsed_in_sw_op;

fprintf(stdout, "-----aml need-----\n");
fprintf(stdout, " : %7.2f ms, inference_time \n",
(float)profile_data.profiling_data.inference_time_us / 1000);
fprintf(stdout, " : %7.2f ms, elapsed_in_hw_op  \n",
(float)profile_data.profiling_data.ext.us_elapsed_in_hw_op / 1000);
fprintf(stdout, " : %7.2f ms, elapsed_in_sw_op  \n",
(float)profile_data.profiling_data.ext.us_elapsed_in_sw_op / 1000);
fprintf(stdout, " : %7.2f ms, elapsed (hw_op+sw_op) \n",
(float)elapsed_hw_sw/1000);
fprintf(stdout, " : %7.2f ms, invoke_time      \n", (float)usVal_inference /
1000);
fprintf(stdout, " : %7.2f  FPS(pm)           \n", (double)1000000 /
(double)profile_data.profiling_data.inference_time_us);
fprintf(stdout, " : %7.2f  FPS(hw+sw)       \n", (double)1000000 /
(double)(elapsed_hw_sw));
fprintf(stdout, " : %7.2f  FPS(invoke) \n", (double)1000000 /
(double)(usVal_inference));
fprintf(stdout, " : %7.2f Mbytes,  dram_read_size
\n", (float)profile_data.profiling_data.dram_read_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes,  dram_write_size
\n", (float)profile_data.profiling_data.dram_write_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes,  BW(Ddr_data)
\n", (double)(profile_data.profiling_data.dram_read_bytes +
profile_data.profiling_data.dram_write_bytes) /
(1024 * 1024));

fprintf(stdout, " : %7.2f Mbytes,  sram_read_bytes
\n", (float)profile_data.profiling_data.sram_read_bytes / (1024 * 1024));
fprintf(stdout, " : %7.2f Mbytes,  sram_write_bytes
\n", (float)profile_data.profiling_data.sram_write_bytes / (1024 * 1024));

```

```
    fprintf(stdout, " : %7.2f Mbytes,   BW(Sram_data)
\n",(double)(profile_data.profiling_data.sram_read_bytes +
profile_data.profiling_data.sram_write_bytes) /
            (1024 * 1024));

    fprintf(stdout, " : %7.2f Mbytes,   memory_usage_bytes
\n",(float)profile_data.profiling_data.memory_usage_bytes / (1024 * 1024));

    aml_util_disableProfile(context, &profile_data);
```

主要关注

invoke_time: npu 耗时
BW(Ddr_data): 带宽占用
memory_usage_bytes: 内存消耗

如果想获取更详细的每一层的信息，那就需要通过设置环境变量来 dump 出 op 文件，以在/data/nn 路径下生成该文件为例，设置环境变量的方法如下

```
export ADLA_DEBUG_OUTPUT_PATH=/data/nn;
export ADLA_ENABLE_DEBUG_AND_PROFILING=1;
设置完环境变量之后再跑 case，就会在/data/nn 路径下生成 op 文件
```

3.6 online 流程介绍

当前建议客户采用离线方式，借助工具将原始模型转换为 ADLA 模型。在转换过程中，可以通过设置环境变量获得中间的量化 TFLite 模型。如果客户不想直接使用工具生成的 ADLA 模型，SDK 也支持在板端将量化后的 TFLite 模型在线转换为 ADLA 模型，但这样会引入额外的时间开销。

以下是一个提供在线转换的示例代码：

```
void* init_network_file(char **argv)
{
    void *qcontext = NULL;
    aml_config config;
    char current_path[256];
    char adla_file_path[256];

    const char *model_path = (const char *)argv[1];

    memset(&config, 0, sizeof(aml_config));
    FILE *fp = fopen(model_path,"rb");
    fseek(fp,0,SEEK_END);
    int model_size = (int)ftell(fp);
    printf("model_size = %d\n",model_size);
```



```
rewind(fp);
config.pdata = (char *)calloc(1,model_size);
fread((void*)config.pdata,1,model_size,fp);
config.length = model_size;
fclose(fp);
config.nbgType = NN_RUNTIME_MEMORY;
config.compiler_args.config.batch_multiplier = 1;
config.compiler_args.compiler_only = 1;

getcwd(current_path, sizeof(current_path));
strcpy(adla_file_path, current_path);
strcat(adla_file_path, "/online_test.adla");
printf("generate adla file is %s\n", adla_file_path);

config.path = adla_file_path;
qcontext = aml_module_create(&config);
if (NULL == qcontext)
{
    printf("aml_module_create fail.\n");
    return NULL;
}

memset(&config, 0, sizeof(aml_config));
config.nbgType = NN_ADLA_FILE;
config.path = adla_file_path;
config.modelType = ADLA_LOADABLE;
config.typeSize = sizeof(aml_config);
qcontext = aml_module_create(&config);
if (NULL == qcontext)
{
    printf("aml_module_create fail.\n");
    return NULL;
}
```

```
if (config.nbgType == NN_ADLA_MEMORY && config.pdata != NULL)
{
    free((void*)config.pdata);
}
return qcontext;
}
```

在线与离线流程的主要区别在于模型加载。在在线流程中，需要调用两次 `aml_module_create` 接口。

第一次调用时，将解析的 TFLite 数据流指针赋给 `config.pdata`，并将 `config.nbgType` 设置为 `NN_RUNTIME_MEMORY`，`config.compiler_args.compiler_only` 设置为 1。调用完成后，会在当前路径下生成 ADLA 文件，文件名可以自行指定，例如这里指定为 `"online_test.adla"`。

第二次调用时，与离线流程相同，但请确保将 `config.path` 设置为刚才生成的 ADLA 文件的路径。